Ideal Function Selection: An Object-Oriented Implementation

Solution to the written assignment in the module “Python Programming”

Submitted by:

Ramin Zandipour

28.09.2026

Repository-Link: https://github.com/RaminZ123/WrittenAssignment

Table of Contents

1. Introduction 3

2. Requirements 3

3. Theoretical Background 3

3.1 Least-Squares Deviation 3

3.2 Deviation Threshold 3

4. Methodology 4

5. Architecture and Design 4

6. Implementation 5

6.1 Database Schema 5

6.2 Function Selection Algorithm 5

6.3 Test Data Mapping 5

6.4 Exception Handling 5

6.5 Unit Testing 5

7. Conclusion 6

8. Additional Task: Version Control Workflow 6

1. Introduction

Based on the written assignment for this module this document and the related code present the implementation of an ideal function selection system using Python with an object-oriented approach. The system processes training data, ideal functions, and test data to identify optimal mappings based on mathematical deviation criteria. The system loads data from CSV files, performs function selection using least-squares deviation, maps test data according to deviation thresholds, and provides visualization of results.

2. Requirements

The system must independently compile a SQLite database via SQLAlchemy and load the following three provided CSV files into separate tables:

- Training data (train.csv)
- Ideal functions (ideal.csv)
- Test data (test.csv)

The implementation must select ideal functions from a set of candidates based on least-squares deviation criteria, map test data to these functions according to deviation thresholds, and provide visualization of the results using the Bokeh library. The solution must follow object-oriented principles with proper class hierarchy and inheritance.

3. Theoretical Background

3.1 Least-Squares Deviation

The function selection algorithm uses the least-squares method to determine optimal mappings between training datasets and ideal functions. For each training dataset Yi (where i = 1, 2, 3, 4), the sum of squared deviations with each ideal function Yj (where j = 1, 2, ..., 50) is calculated using the sum of squared deviations formula. The ideal function with the minimum sum of squared deviations is selected for each training dataset.

3.2 Deviation Threshold

The test data mapping uses a deviation criterion where a test point (x, y) maps to an ideal function if the absolute deviation between the test point's y-value and the ideal function's y-value at the same x-coordinate is less than or equal to the maximum deviation between the corresponding training dataset and its selected ideal function, multiplied by $\sqrt{2}$.

4. Methodology

The methodology follows a structured pipeline implemented through object-oriented methods:

1. **Data Loading**: Training data, ideal functions, and test data are loaded from CSV files into a SQLite database
2. **Function Selection**: For each training dataset, the ideal function with minimum sum of squared deviations is selected
3. **Maximum Deviation Calculation**: Maximum deviations between training datasets and their selected ideal functions are calculated
4. **Test Data Mapping**: Test points are evaluated against selected ideal functions according to deviation criteria
5. **Visualization**: Results are visualized using interactive plots with hover tooltips

Libraries used:

1. **NumPy:** Used for the least-squares deviation calculations and mathematical operations in the function selection algorithm
2. **Pandas:** Used for reading the CSV files, creating DataFrames for data manipulation and transferring data between files and database tables
3. **SQLAlchemy:** Used for creating and managing the SQLite database, establishing database connections, and handling all database table operations
4. **Bokeh:** Used for generating interactive plots with hover tooltips to visualize the relationship between test data and selected ideal functions

5. Architecture and Design

The system employs a hierarchical class structure:

- **DatabaseManager**: Base class providing database functionality
- **DataProcessor**: Inherits from DatabaseManager, adds data processing capabilities
- **DataVisualizer**: Inherits from DatabaseManager, adds visualization capabilities
- **IdealFunctionApp**: Coordinates the overall application workflow

6. Implementation

6.1 Database Schema

The system utilizes a SQLite database with three tables:

- **TrainingData**: Stores training data with columns X, Y1, Y2, Y3, Y4
- **IdealFunctions**: Stores ideal functions with columns X, Y1, Y2, ..., Y50
- **TestDataMapping**: Stores test data mapping results with columns id, X, Y, No_of_ideal_func, Delta_Y

6.2 Function Selection Algorithm

The select_best_ideal_functions method implements the function selection algorithm by comparing each training dataset with all ideal functions using the least-squares criterion. For each training dataset, the algorithm calculates the sum of squared deviations with each ideal function and selects the function with the minimum deviation.

6.3 Test Data Mapping

The map_test_data method implements the test data mapping algorithm by processing each test point against the selected ideal functions. Each test point is evaluated according to the deviation criterion, and if it meets the threshold, it is mapped to the corresponding ideal function with the calculated deviation.

6.4 Exception Handling

The system defines the following custom exception classes to handle specific error scenarios:

- **DataLoadError**: Raised when data loading from CSV files fails
- **FunctionSelectionError**: Raised when the function selection algorithm encounters errors

These custom exceptions provide more specific error information than generic exceptions, allowing for more precise error handling and user feedback.

6.5 Unit Testing

The system includes unit tests to verify the correctness of all major components. The unit tests are implemented in a separate file (test_ideal_function_app.py) following standard Python testing practices.

The unit tests focus on verifying basic system functionality:

- Class initialization for core components (DatabaseManager, DataProcessor, DataVisualizer, IdealFunctionApp)
- Method existence verification for key classes
- Database connection establishment

7. Conclusion

The system successfully processes training data, ideal functions, and test data according to the specified methodology. The function selection algorithm identifies optimal ideal functions for each training dataset based on minimum sum of squared deviations. The test data mapping algorithm successfully maps test points to selected ideal functions according to deviation criteria. The visualization component provides interactive plots with hover tooltips for detailed examination of results.

8. Additional Task: Version Control Workflow

To work with the project's version control system, the following Git commands are used to clone the develop branch and set up the local development environment:

Clone the repository and switch to the develop branch:

```
git clone --branch develop <repository-url>
cd <repository-directory>
```

Verify the current branch:

```
git branch
```

When implementing new functionality, create a new feature branch from develop:

```
git checkout -b feature/new-functionality
```

Then make changes to the code or implement new functionality and stage the changes for commit:

```
git add .
```

Commit the changes with a descriptive message:

```
git commit -m "Implemented new functionality"
```

Push the feature branch to the remote repository:

```
git push origin feature/new-functionality
```

After implementing and testing the new functionality, the code is integrated into the main development branch through a pull request on the Git platform, which is then reviewed by team members. After approval, the pull request is merged into the develop branch.

ideal_function_app.py

```python
"""
Ideal Function Selection and Test Data Mapping Application

"""

import pandas as pd
import numpy as np
from sqlalchemy import create_engine, Column, Float, Integer
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker
from bokeh.plotting import figure
from bokeh.plotting import show
from bokeh.io import output_file
from bokeh.models import ColumnDataSource
from bokeh.models import HoverTool
from bokeh.layouts import gridplot
import os
import sys

# Create a shared Base
Base = declarative_base()


class DataLoadError(Exception):
    """Exception raised when data loading fails."""

    pass

class FunctionSelectionError(Exception):
    """Exception raised when function selection fails."""

    pass


class TrainingData(Base):
    """Table for storing training data."""

    __tablename__ = 'training_data'

    X = Column(Float, primary_key=True)
    Y1 = Column(Float)
    Y2 = Column(Float)
    Y3 = Column(Float)
    Y4 = Column(Float)


class IdealFunctions(Base):
    """Table for storing ideal functions."""

    __tablename__ = 'ideal_functions'

    X = Column(Float, primary_key=True)
    # 50 ideal functions
    for i in range(1, 51):
        locals()['Y{}'.format(i)] = Column(Float)


class TestDataMapping(Base):
    """Table for storing test data mapping results."""

    __tablename__ = 'test_data_mapping'

    id = Column(Integer, primary_key=True, autoincrement=True)
    X = Column(Float)
    Y = Column(Float)
```

```
    No_of_ideal_func = Column(Integer)
    Delta_Y = Column(Float)


class DatabaseManager:
    """Base class for managing database operations."""

    def __init__(self, db_path='database.db'):
        """Initialize the database manager."""

        self.db_path = db_path
        self.engine = create_engine('sqlite:///{}'.format(db_path))
        self.Base = Base
        self.Session = sessionmaker(bind=self.engine)

    def create_tables(self):
        """Create all tables defined in the Base metadata."""

        self.Base.metadata.create_all(self.engine)

    def get_session(self):
        """Get a new database session."""

        return self.Session()


class DataProcessor(DatabaseManager):
    """Class for processing data, including loading, selecting ideal functions, and mapping test data."""

    def __init__(self, db_path='database.db'):
        """Initialize the data processor."""

        super().__init__(db_path)
        self.TrainingData = TrainingData
        self.IdealFunctions = IdealFunctions
        self.TestDataMapping = TestDataMapping
        self.create_tables()

    def load_training_data(self, training_data_file_path):
        """
        Load training data from the CSV file into the database.

        Args:
            training_data_file_path (str): Path to the training data CSV file.

        Raises:
            DataLoadError: If data loading fails.
        """
        try:
            # Load data from file
            df = pd.read_csv(training_data_file_path)

            df.columns = ['X', 'Y1', 'Y2', 'Y3', 'Y4']

            # Load to database
            df.to_sql('training_data', self.engine, if_exists='replace', index=False)

        except Exception as e:
            raise DataLoadError("Failed to load training data: {}".format(str(e)))

    def load_ideal_functions(self, ideal_functions_file_path):
        """
        Load ideal functions from the CSV file into the database.

        Args:
            ideal_functions_file_path (str): Path to the ideal functions CSV file.

```

```
        Raises:
            DataLoadError: If data loading fails.
        """
        try:
            # Load data from file
            df = pd.read_csv(ideal_functions_file_path)

            df.columns = ['X'] + ['Y{}'.format(i) for i in range(1, 51)]

            # Load to database
            df.to_sql('ideal_functions', self.engine, if_exists='replace', index=False)

        except Exception as e:
            raise DataLoadError("Failed to load ideal functions: {}".format(str(e)))

    def select_best_ideal_functions(self):
        """
        Select the best ideal functions for each training dataset.

        Returns:
            list: List of indices of the selected ideal functions.

        Raises:
            FunctionSelectionError: If function selection fails.
        """
        try:

            session = self.get_session()

            # Get training data
            training_data = pd.read_sql_table('training_data', self.engine)

            # Get ideal functions
            ideal_functions = pd.read_sql_table('ideal_functions', self.engine)

            session.close()

            selected_function_indices = []

            # For each training dataset initialize minimum sum of squares to infinity
            for training_dataset_index in range(1, 5):
                min_sum_squares = float('inf')
                best_function_index = -1

                # For each ideal function calculate sum of squared deviations
                for ideal_function_index in range(1, 51):
                    sum_of_squared_deviations = np.sum( (training_data['Y{}'.format(training_dataset_index)] -
ideal_functions['Y{}'.format(ideal_function_index)])**2)

                    # Update if this function has a smaller sum of squares
                    if sum_of_squared_deviations < min_sum_squares:
                        min_sum_squares = sum_of_squared_deviations
                        best_function_index = ideal_function_index

                # Add the best function index to the results
                selected_function_indices.append(best_function_index)

            return selected_function_indices

        except Exception as e:
            raise FunctionSelectionError("Failed to select ideal functions: {}".format(str(e)))

    def calculate_maximum_deviations(self, selected_function_indices):
        """
        Calculate maximum deviations between training data and selected ideal functions.

        Args:
```

```
            selected_function_indices (list): List of indices of the selected ideal functions.

        Returns:
            list: List of maximum deviations for each training dataset.

        Raises:
            DataLoadError: If calculation fails.
        """
        try:
            # Get training data
            session = self.get_session()
            training_data = pd.read_sql_table('training_data', self.engine)

            # Get ideal functions
            ideal_functions = pd.read_sql_table('ideal_functions', self.engine)

            # Close session
            session.close()

            # Calculate maximum deviation for each training dataset and its ideal function
            maximum_deviations = []

            # For each training dataset and its selected ideal function calculate the deviations
            for training_dataset_index, ideal_function_index in enumerate(selected_function_indices):
                deviations = np.abs(training_data['Y{}'.format(training_dataset_index + 1)] - ideal_functions['Y{}'.format(ideal_function_index)])
                # Get maximum deviation
                maximum_deviation = np.max(deviations)
                # Add to results
                maximum_deviations.append(maximum_deviation)

            return maximum_deviations

        except Exception as e:
            raise DataLoadError("Failed to calculate maximum deviations: {}".format(str(e)))

    def map_test_data(self, test_data_file_path, selected_function_indices, maximum_deviations):
        """
        Map test data to the selected ideal functions.

        Args:
            test_data_file_path (str): Path to the test data CSV file.
            selected_function_indices (list): List of indices of the selected ideal functions.
            maximum_deviations (list): List of maximum deviations for each training dataset.

        Returns:
            DataFrame: DataFrame containing the mapping results.

        Raises:
            DataLoadError: If data loading or mapping fails.
        """
        try:
            # Get ideal functions
            session = self.get_session()
            ideal_functions = pd.read_sql_table('ideal_functions', self.engine)

            # Load test data
            test_data = pd.read_csv(test_data_file_path)

            test_data.columns = ['X', 'Y']

            # Map test data
            mapping_results = []

            # For each test point
            for _, test_point in test_data.iterrows():
                test_x_value = test_point['X']
```

```
                test_y_value = test_point['Y']

                # For each selected ideal function find the ideal function value at test_x_value
                for training_dataset_index, ideal_function_index in enumerate(selected_function_indices):

                    ideal_function_row = ideal_functions[ideal_functions['X'] == test_x_value]

                    if not ideal_function_row.empty:
                        ideal_y_value = ideal_function_row.iloc[0, ideal_function_index]
                        deviation = np.abs(test_y_value - ideal_y_value)

                        # Check if deviation is within threshold
                        if deviation <= maximum_deviations[training_dataset_index] * np.sqrt(2):
                            mapping_results.append({
                                'X': test_x_value,
                                'Y': test_y_value,
                                'No_of_ideal_func': ideal_function_index,
                                'Delta_Y': deviation
                            })
                            break

            # Save results to database
            results_df = pd.DataFrame(mapping_results)
            results_df.to_sql('test_data_mapping', self.engine, if_exists='replace', index=False)

            # Close session
            session.close()

            return results_df

        except Exception as e:
            raise DataLoadError("Failed to map test data: {}".format(str(e)))


class DataVisualizer(DatabaseManager):
    """Class for visualizing data using Bokeh."""

    def __init__(self, db_path='database.db'):
        """Initialize the data visualizer."""

        super().__init__(db_path)

    def visualize_data(self, selected_function_indices):
        """
        Visualize training data, test data, and selected ideal functions.

        Args:
            selected_function_indices (list): List of indices of the selected ideal functions.
        """

        session = self.get_session()

         # Get data from database
        training_data = pd.read_sql_table('training_data', self.engine)
        ideal_functions = pd.read_sql_table('ideal_functions', self.engine)
        test_mapping = pd.read_sql_table('test_data_mapping', self.engine)

        session.close()

        plot_grid = []

        # For each training dataset create a new plot
        for training_dataset_index in range(1, 5):
            plot_title = "Training Dataset {} and Ideal Function {}".format(training_dataset_index,
selected_function_indices[training_dataset_index-1])
            current_plot = figure(title=plot_title, x_axis_label='X', y_axis_label='Y')

```

```
            current_plot.scatter(
                x=training_data['X'],
                y=training_data['Y{}'.format(training_dataset_index)], size=8, color='blue', alpha=0.7, legend_label='Training Data')

            # Plot ideal function
            ideal_function_index = selected_function_indices[training_dataset_index-1]
            current_plot.line(
                ideal_functions['X'], ideal_functions['Y{}'.format(ideal_function_index)], line_width=2, color='red', legend_label='Ideal Function')

            # Plot test data mapping
            test_data_for_function = test_mapping[test_mapping['No_of_ideal_func'] == ideal_function_index]
            if not test_data_for_function.empty:
                test_data_source = ColumnDataSource(data={'x': test_data_for_function['X'], 'y': test_data_for_function['Y'], 'Delta_Y': test_data_for_function['Delta_Y']})
                current_plot.scatter(x='x', y='y', source=test_data_source,size=8, color='green', alpha=0.7, legend_label='Test Data')

            # Add hover tool for deviation
            hover = HoverTool()
            hover.tooltips = [("X", "@x"), ("Y", "@y"), ("Deviation", "@Delta_Y")]
            current_plot.add_tools(hover)

            plot_grid.append(current_plot)

        # Create a grid of plots
        grid = gridplot([plot_grid[:2], plot_grid[2:]])

        # Show the plot in a browser
        output_file("ideal_functions.html")
        show(grid)


class IdealFunctionApp:
    """Main application class for the ideal function assignment."""

    def __init__(self, db_path='database.db'):
        """Initialize the application."""
        self.db_path = db_path
        self.data_processor = DataProcessor(db_path)
        self.data_visualizer = DataVisualizer(db_path)

    def run_application(self, training_data_file, ideal_functions_file, test_data_file):
        """
        Run the application.

        Args:
            training_data_file (str): Path to the training data CSV file.
            ideal_functions_file (str): Path to the ideal functions CSV file.
            test_data_file (str): Path to the test data CSV file.
        """
        try:
            # Load data
            print("Loading training data...")
            self.data_processor.load_training_data(training_data_file)

            print("Loading ideal functions...")
            self.data_processor.load_ideal_functions(ideal_functions_file)

            # Select best ideal functions
            print("Selecting best ideal functions...")
            selected_function_indices = self.data_processor.select_best_ideal_functions()
            print("Selected ideal functions: {}".format(selected_function_indices))

            # Calculate maximum deviations
            print("Calculating maximum deviations...")
            maximum_deviations = self.data_processor.calculate_maximum_deviations(selected_function_indices)
```

```
            # Map test data
            print("Mapping test data...")
            mapping_results = self.data_processor.map_test_data(test_data_file, selected_function_indices, maximum_deviations)
            print("Mapped {} test data points.".format(len(mapping_results)))

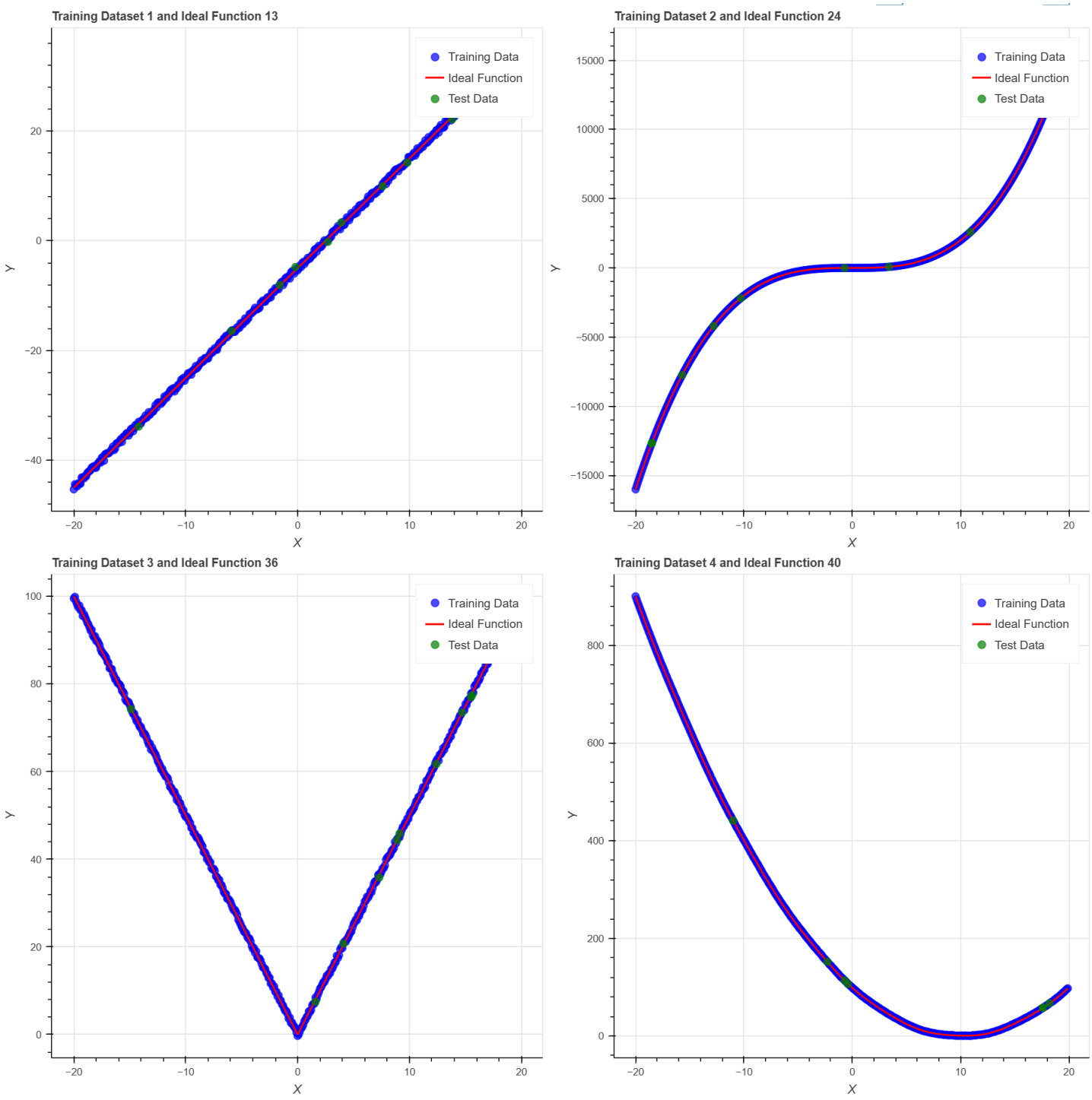
            # Visualize data
            print("Visualizing data...")
            self.data_visualizer.visualize_data(selected_function_indices)

            print("Application completed successfully.")

        except Exception as e:
            print("Error: {}".format(str(e)))

# Main execution
if __name__ == "__main__":
    training_data_file = "train.csv"
    ideal_functions_file = "ideal.csv"
    test_data_file = "test.csv"

    # Create and run the application
    app = IdealFunctionApp()
    app.run_application(training_data_file, ideal_functions_file, test_data_file)
```


Training Dataset 1 and Ideal Function 13
Training Data
Ideal Function
Test Data
X
Y
Training Dataset 2 and Ideal Function 24
Training Data
Ideal Function
Test Data
X
Y
Training Dataset 3 and Ideal Function 36
Training Data
Ideal Function
Test Data
X
Y
Training Dataset 4 and Ideal Function 40
Training Data
Ideal Function
Test Data
X
Y

test_ideal_function_app.py

```python
import tempfile
import unittest

from ideal_function_app import DatabaseManager, DataProcessor, DataVisualizer, IdealFunctionApp

class TestIdealFunctionApp(unittest.TestCase):
    """Unit tests for the Ideal Function App."""

    def setUp(self):
        """Set up test fixtures before each test method."""

        self.temp_db = tempfile.NamedTemporaryFile(suffix='.db', delete=False)
        self.temp_db.close()
        self.db_path = self.temp_db.name

    def tearDown(self):
        """Clean up after each test method."""

        import os
        try:
            os.unlink(self.db_path)
        except:
            pass

    def test_database_manager_initialization(self):
        """Test DatabaseManager initialization."""

        db_manager = DatabaseManager(self.db_path)
        self.assertEqual(db_manager.db_path, self.db_path)

    def test_data_processor_initialization(self):
        """Test DataProcessor initialization."""

        processor = DataProcessor(self.db_path)
        self.assertEqual(processor.db_path, self.db_path)

    def test_data_visualizer_initialization(self):
        """Test DataVisualizer initialization."""

        visualizer = DataVisualizer(self.db_path)
        self.assertEqual(visualizer.db_path, self.db_path)

    def test_ideal_function_app_initialization(self):
        """Test IdealFunctionApp initialization."""

        app = IdealFunctionApp()
        self.assertIsNotNone(app.data_processor)
        self.assertIsNotNone(app.data_visualizer)

    def test_database_connection(self):
        """Test database connection."""

        db_manager = DatabaseManager(self.db_path)
        session = db_manager.get_session()
        self.assertIsNotNone(session)
        session.close()

    def test_data_processor_methods_exist(self):
        """Test that DataProcessor has the required methods."""

        processor = DataProcessor(self.db_path)

        self.assertTrue(hasattr(processor, 'load_training_data'))
        self.assertTrue(hasattr(processor, 'load_ideal_functions'))
        self.assertTrue(hasattr(processor, 'select_best_ideal_functions'))
```

```
66          self.assertTrue(hasattr(processor, 'calculate_maximum_deviations'))
67          self.assertTrue(hasattr(processor, 'map_test_data'))
68
69      def test_data_visualizer_methods_exist(self):
70          """Test that DataVisualizer has the required methods."""
71
72          visualizer = DataVisualizer(self.db_path)
73
74          self.assertTrue(hasattr(visualizer, 'visualize_data'))
75
76      def test_ideal_function_app_methods_exist(self):
77          """Test that IdealFunctionApp has the required methods."""
78
79          app = IdealFunctionApp()
80          self.assertTrue(hasattr(app, 'run_application'))
81
82
83  if __name__ == '__main__':
84      unittest.main()
```